In [60]:
```
# Install once if needed:
# pip install pandas numpy matplotlib scikit-learn tensorflow

# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from tensorflow.keras import layers, models
```

In [61]:
```
# 2. Load dataset
df = pd.read_csv("creditcard.csv")

print(df.head())
print("Dataset shape:", df.shape)
```

```
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
Dataset shape: (284807, 31)
```

In [62]:
```python
# 3. Separate features and labels
X = df.drop("Class", axis=1)
y = df["Class"]

# 4. Split into training and testing data
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [63]:
```python
# 5. Keep only genuine transactions for training
x_train = x_train[y_train == 0]

# 6. Scale features
scaler = MinMaxScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

In [65]:
```
# 7. Build Autoencoder
encoder = models.Sequential([
    layers.Input(shape=(x_train.shape[1],)),
    layers.Dense(20, activation="relu"),
    layers.Dense(14, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(4, activation="relu")
])

decoder = models.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(8, activation="relu"),
    layers.Dense(14, activation="relu"),
    layers.Dense(20, activation="relu"),
    layers.Dense(x_train.shape[1], activation="sigmoid")
])

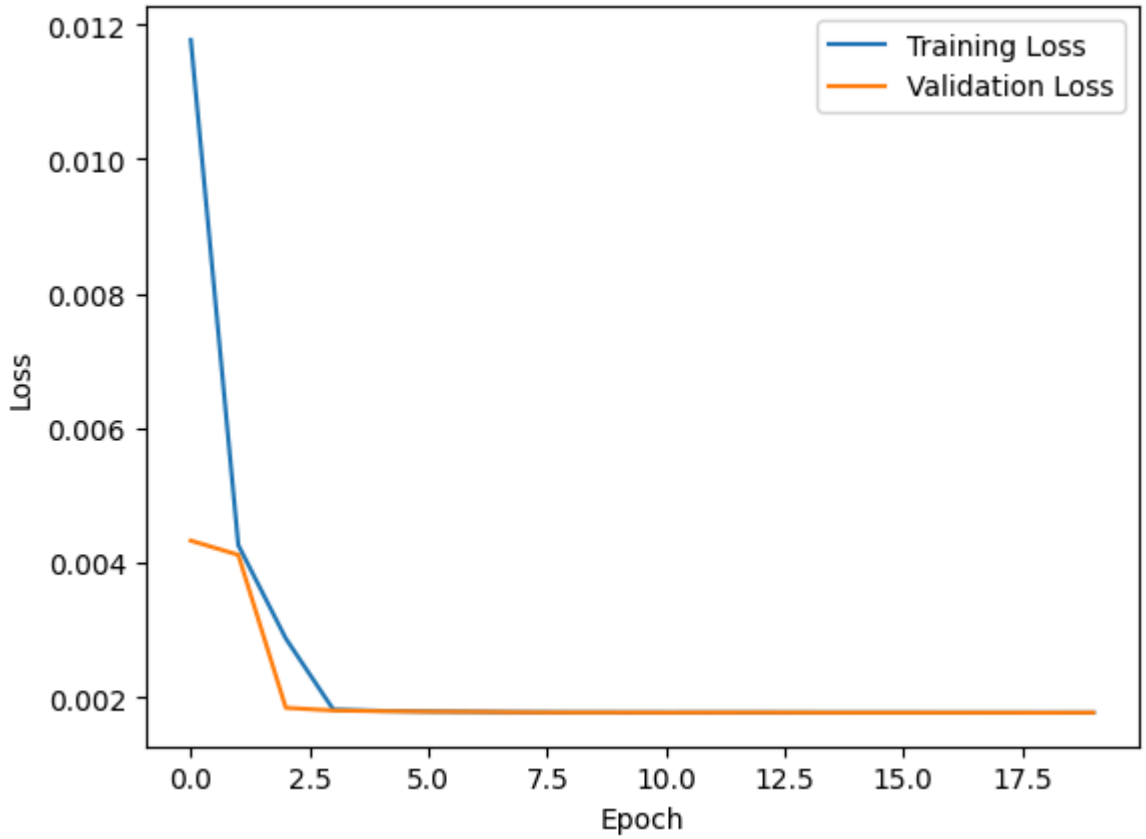
autoencoder = models.Sequential([encoder, decoder])
```

In [66]:
```
# 8. Compile model
autoencoder.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)
```

In [67]:
```
# 9. Train autoencoder on genuine transactions
history = autoencoder.fit(
    x_train, x_train,
    epochs=20,
    batch_size=512,
    validation_split=0.1,
    shuffle=True
)
```

```
Epoch 1/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.0118 - mae: 0.0638 - val_loss: 0.0043 - val_mae: 0.0345
Epoch 2/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0043 - mae: 0.0347 - val_loss: 0.0041 - val_mae: 0.0357
Epoch 3/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0029 - mae: 0.0330 - val_loss: 0.0018 - val_mae: 0.0261
Epoch 4/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0257 - val_loss: 0.0018 - val_mae: 0.0255
Epoch 5/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0254 - val_loss: 0.0018 - val_mae: 0.0254
Epoch 6/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0253 - val_loss: 0.0018 - val_mae: 0.0253
Epoch 7/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0252 - val_loss: 0.0018 - val_mae: 0.0253
Epoch 8/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0251 - val_loss: 0.0018 - val_mae: 0.0252
Epoch 9/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0251 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 10/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0251 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 11/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0251 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 12/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0251 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 13/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 14/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 15/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 16/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0251
Epoch 17/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0250
Epoch 18/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0250
Epoch 19/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0250
Epoch 20/20
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0018 - mae: 0.0250 - val_loss: 0.0018 - val_mae: 0.0251
```

In [68]:
```
# 10. Plot training loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



In [69]:
```
# 11. Calculate reconstruction errors on training data
train_reconstructed = autoencoder.predict(x_train, verbose=0)
train_errors = np.mean(np.square(x_train - train_reconstructed), axis=1)

# Set threshold using mean + standard deviation
```

```
threshold = np.mean(train_errors) + np.std(train_errors)
print("Reconstruction Error Threshold:", threshold)
```

```
Reconstruction Error Threshold: 0.0034122267111192316
```

In [70]:
```
# 12. Predict fraud on test data
test_reconstructed = autoencoder.predict(x_test, verbose=0)
test_errors = np.mean(np.square(x_test - test_reconstructed), axis=1)

# Error above threshold = fraud
y_pred = (test_errors > threshold).astype(int)
```

In [71]:
```
# 13. Evaluate model
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Accuracy: 0.9259506337558372

Confusion Matrix:
[[52657  4207]
 [   11    87]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.93      0.96     56864
           1       0.02      0.89      0.04        98

    accuracy                           0.93     56962
   macro avg       0.51      0.91      0.50     56962
weighted avg       1.00      0.93      0.96     56962
```

In [ ]:
```

```